

Robot Continual Learning With Knowledge Sharing and Model Acceleration for Sequential Assembly Tasks

Ligang Jin , Chaoqun Wang , Zhiwei Cui , Xincheng Tian , Yibin Li , *Member, IEEE*,
Fuchun Sun , *Fellow, IEEE*, and Rui Song , *Senior Member, IEEE*

I. INTRODUCTION

Abstract—Within the domain of robot assembly manipulation, contemporary skill-learning approaches exhibit significant generalization capabilities, enabling them to adapt to a broad spectrum of tasks and dynamic environmental conditions. However, these methods suffer from catastrophic forgetting when applied to various robot manipulation tasks, leading to suboptimal performance and inadequate adaptability in real-world scenarios. To this end, a continual learning strategy that incorporates both knowledge sharing and model acceleration is proposed for multiple robot assembly tasks. First, a common experience pool is established that enhances data efficiency by selectively revisiting past task experiences during the learning process. This mechanism mitigates forgetting by preserving critical knowledge from previous tasks. Second, incorporating the effects of the old model, the new model can converge more rapidly. The weights of the old and new models are learned through interaction with the environment. Experiments on various assembly tasks are conducted to verify the continuous memory retention and efficient learning capabilities of the proposed method in both simulation and real-world environments.

Index Terms—Continual learning, knowledge sharing, model acceleration, robot assembly.

TOWARD the manipulation tasks [1], [2], robots are playing more important roles in industrial assembly, home service, and other applications [3]. Unlike traditional control methods or preprogramming, recent studies indicate that robots can acquire abundant skills similar to humans. For example, in assembly tasks, robots are capable of performing peg-in-hole assembly with different numbers of pegs and holes, different shapes, and different tolerances [4], [5]. However, existing methods suffer from catastrophic forgetting when confronted with diverse manipulation tasks, limiting their adaptability to dynamic scenarios. Traditional approaches, which rely on precise physical modeling or preprogrammed strategies, tend to excel at single-task execution but lack the ability to generalize across multiple tasks. Conversely, most skill learning methods, though capable of handling complex tasks, often neglect knowledge retention and transfer between tasks. This article proposes a continual learning framework integrating knowledge sharing and model acceleration mechanisms. By synergistically combining historical model knowledge and experience replay, it effectively mitigates forgetting while accelerating new skill acquisition.

The accurate physical models of assembly objects have been established in robot assembly tasks recently. By establishing an equivalent model of multiple peg-in-hole assembly, the instantaneous center of rotation (ICR) planning is proposed to enhance force tracking and speed performance [6]. Furthermore, combining passive and active compliant strategies, Su et al. [7] construct the high-dimensional configuration space and its subspace of the assembly, enabling robot precision assembly. The vision/force hybrid control method [8] enables precise assembly of tiny parts, but the positioning accuracy of vision systems is limited and lacks sufficient generalization capability across diverse assembly objects. Force control, on the other hand, offers smooth operation and ensures safe protection [9]. In addition, some precision assembly tasks involving narrow clearances also strictly rely on the acquisition of force sensing information [10], [11]. However, when relying solely on force control, the robot lacks the ability to determine the direction for the next step during the search or insertion process. Therefore, effectively handling search and insertion tasks still depends on robust and reliable skill-learning strategies.

Received 1 November 2025; revised 7 January 2026; accepted 15 February 2026. Date of publication 23 March 2026; date of current version 24 June 2026. This work was supported in part by the National Key Research and Development Plan of China under Grant 2023YFB4705002, in part by China Postdoctoral Science Foundation under Grant 2025M781652, and in part by the General Program of the National Natural Science Foundation of China under Grant 62373225. (Corresponding author: Zhiwei Cui.)

Ligang Jin and Zhiwei Cui are with the School of Artificial Intelligence, Shandong University, Jinan 250101, China (e-mail: lgjin@sdu.edu.cn; czw_2025@sdu.edu.cn; czw16@tsinghua.org.cn).

Chaoqun Wang, Xincheng Tian, Yibin Li, and Rui Song are with the School of Control Science and Engineering, Shandong University, Jinan 250061, China (e-mail: chaoqunwang@sdu.edu.cn; txch@sdu.edu.cn; liyb@sdu.edu.cn; rsong@sdu.edu.cn).

Fuchun Sun is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China, and also with the School of Artificial Intelligence, Shandong University, Jinan 250101, China (e-mail: fcsun@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TIE.2026.3670234

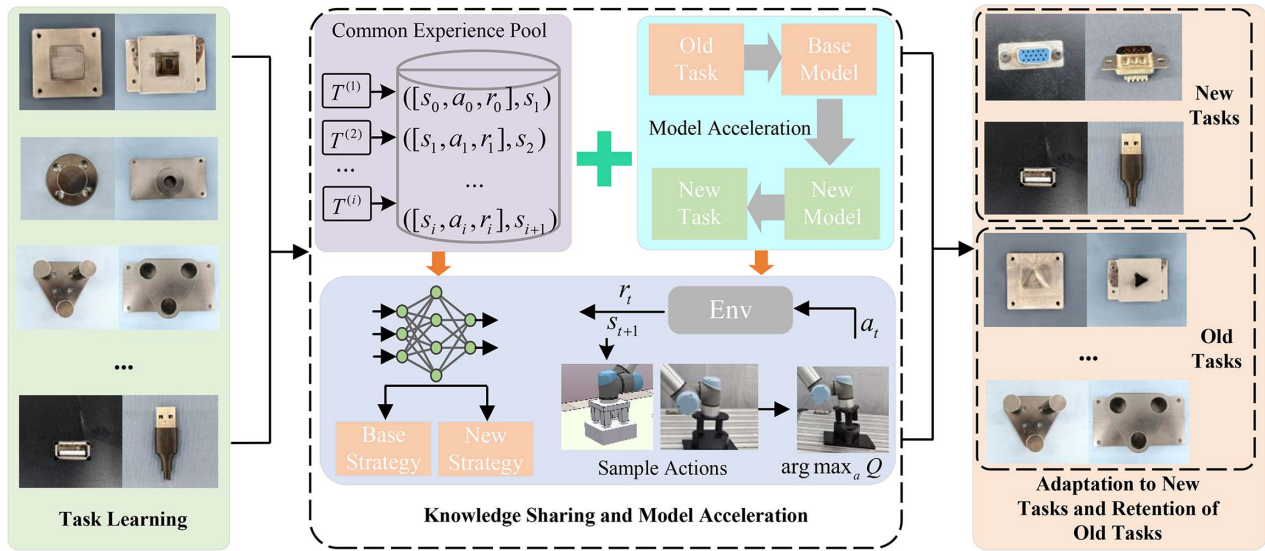


Fig. 1. Schematic of the robot continual learning process. As the robot learns each task, $T^{(1)}$, $T^{(2)}$, $\dots$, $T^{(i)}$, it stores data from interactions with the environment in a common experience pool. When learning new tasks, the robot samples and reviews the knowledge in the common experience pool. Furthermore, the model acceleration module is incorporated into the learning process for new tasks, leveraging skills learned from previous tasks to guide new task learning. Ultimately, the model not only rapidly learns new tasks but also retains a strong memory for previous tasks.

In contrast, skill learning methods such as reinforcement learning and transfer learning are also used for robot sequential assembly tasks. Reinforcement learning combined with fuzzy logic [12] has been used to perform dual peg-in-hole assembly, predicting contact force information during the assembly process. In addition, skill transfer learning is frequently employed to address the issue of skill learning across multiple tasks. By collecting the source domain environment kinetic models [13], the target domain kinetic model can be established, enabling the execution of optimal actions in the target domain. Furthermore, knowledge from daily activities can also be transferred to industrial assembly scenarios [14], thereby enabling automobile robots to assist human. Compared to traditional control methods [15], transfer learning achieves task generalization. However, transfer learning is typically a one-way transfer process, and once the transfer is complete, the performance of the model on previous tasks is usually disregarded.

In sequential assembly tasks, catastrophic forgetting poses a significant challenge for robots. Catastrophic forgetting is the phenomenon of a neural network experiencing performance degradation at previously learned concepts when trained sequentially on learning new concepts. Continual learning refers to robots learn a series of skills in a specific order without forgetting previous tasks [16]. As illustrated in Fig. 1, this article introduces a robot continual learning method that integrates knowledge sharing and model acceleration to address sequential assembly tasks. When robots perform different assembly tasks, different types of data are generated during each task interaction, which contains assembly process information for every type of task encountered. By utilizing this knowledge, the learned strategies can maintain their adaptability to previous tasks, thereby enhancing their continual learning ability. This approach enables robots to learn a family of manipulation tasks progressively without forgetting previous tasks. Compared to existing continual learning methods, the method proposed in

this article establishes a common experience pool. This strategy selectively stores interaction information from historical tasks, allowing the model to effectively recall prior knowledge when learning new tasks. Furthermore, this method combines the advantages of both data- and model-driven approaches, enhancing the ability to retain knowledge from old tasks while also accelerating the learning speed for new tasks. The main contributions of this article are as follows.

- 1) A knowledge sharing mechanism for robot assembly is proposed to solve the robot's forgetting of previously learned tasks. By reviewing the knowledge of old tasks, catastrophic forgetting of old tasks can be mitigated.
- 2) The old model is used to guide the learning of the new task, which can accelerate the learning process. It relearns based on existing common features to quickly adapt to new tasks and does not need to learn from scratch.
- 3) The proposed method demonstrates excellent versatility and adaptability across a variety of complex assembly tasks. More importantly, it possesses excellent cross-domain transfer capabilities, enabling the stable transfer of strategies learned in simulation to the robot systems in the real-physical world.

The rest of this article is organized as follows. Section II introduces the related work. The methods of this article are detailed in Section III. The details of the assembly simulation platform and the experiment results are shown in Section IV. Section V provides the performance of the real-world experiments. Finally, Section VI concludes this article.

II. RELATED WORK

A. Skill Learning for Assembly

In recent work, reinforcement learning is frequently used to enable autonomous learning of robot assembly skills. Combining the model-based reinforcement learning, Liu

et al. [17] use the real data recorded during the real-world assembly process to generate a fake environment that can be interacted with. Nevertheless, they lack the predictions of the assembly environment, such as the assembly tolerance or the shape of the assembly objects. Zhu et al. [18] propose a hierarchical deep reinforcement learning framework for real-time assembly planning of large-scale prefabricated components, demonstrating significant efficiency improvements in construction automation. Hou et al. [19] propose a hierarchical reinforcement learning framework to address the exploration efficiency limitations of existing reinforcement learning algorithms, which significantly improves the learning efficiency of contact-rich assembly tasks in real-world environments. Shi et al. [20] introduce a residual reinforcement learning method, attempting to inject prior information into the reinforcement learning algorithm to accelerate the training process rather than starting with random exploration from scratch. However, their approach assumes fixed environmental parameters and fails to address catastrophic forgetting in multitask sequential learning.

Furthermore, imitation learning has made significant progress in addressing robot skill learning problems. Utilizing a Gaussian mixture model, Song et al. [21] fit human teaching data and apply the compliant control method to accomplish the assembly process when environment parameters are inaccurate. Learning from human demonstration videos [22] has proven to be an effective method for robots to learn assembly trajectories. Recent research on demonstrations [23] extends imitation learning to sequential tasks, but it relies heavily on continuous human guidance. Additionally, the training time for hypernetworks grows substantially as the number of tasks increases.

B. Continual Learning for Sequential Assembly

Methods based on experience replay [24] and regularization [25] achieve continual learning from the perspectives of data and model, respectively. Continual learning based on experience replay [26] can alleviate catastrophic forgetting, but it is not sufficiently guided in learning new tasks. Adversarial continual learning [27] requires generating adversarial samples, which increase computational costs. In addition, the switching strategies that allow them to effectively combine complex skills with existing primitive skills are also used to perform sequential tasks [28]. Lee et al. [29] combine previously learned skills sequentially through skill chains learning to realize long-horizon and hierarchical manipulation tasks. However, both of the above methods lack solutions to the catastrophic forgetting problem in multiple tasks learning.

Methods based on task representations [30] use unlabeled demonstrations to realize task inference, which requires prior demonstration or the learning of unknown tasks. Schwarz et al. [31] add the connection between the teacher's and student's policy but do not represent the knowledge explicitly. For diverse tasks, it needs reinitializing the parameters of the networks, which is cumbersome or even impossible for abundant tasks. Zhao et al. [32] propose a model acceleration method where the assembly strategy is learned through a maximum

entropy reinforcement learning framework, allowing for skill transfer between tasks while ensuring exploration efficiency. However, their approach focuses on utilizing transfer data and does not adequately leverage previously learned models. Furthermore, progressive neural networks (PNNs) [33] are not susceptible to forgetting and can exploit prior knowledge through lateral connections with previously learned features. Nevertheless, PNNs will gradually increase the number of network layers, which will increase the computational and storage burden. "Memory-based parameter adaptation" [34] method combines architecture growth to prevent forgetting of task-specific skills and an experience replay approach to preserve shared skills. However, this approach relies on fixed replay and lacks validation on manipulation tasks.

Continual learning is often combined with other skill learning methods to enhance the efficiency of robot manipulation task learning. Imitation learning is typically integrated with continual learning [35] to enable robots to continually learn new tasks sequentially, reducing the burden of imitation learning and accelerating the process of acquiring new skills. Maltoni et al. [36] focus on single-incremental-task scenarios via regularization but lack task difficulty measurement and environment prediction. Whereas, previous continual learning methods often do not consider the learning sequence of various tasks nor do they adequately address how to combine the models and data of encountered tasks.

III. METHOD

A. Insertion Process Staged Analysis

The peg-in-hole assembly manipulation process is a segmented, continuous motion subject to various constraints such as geometry, force, and area. It is a compliant and continuous process of discrete contact states. During multiple peg-in-hole assembly insertion processes, phases of jamming and pose adjustment occur. Due to inherent accuracy limitations of the robot and sensor errors, there is typically a position and orientation offset between the peg and hole. During the assembly process, jamming and wedging phenomena will occur, resulting in significant contact forces and torques. Therefore, collaborative control of assembly strategy and force control is essential. Force control aims to protect the workpiece, prevent excessive contact forces, and expedite the assembly process. The assembly strategy adjusts the robot's pose to ensure smooth assembly.

The entire assembly process is depicted in Fig. 2. Robot assembly tasks can be described as a Markov decision process (MDP), which contains five elements S, A, P, R, γ . In a robot assembly task, S represents the contact state between the robot end and the environment, and A represents the action the robot will execute in the next step. P is the state transition probability, which describes the probability of the robot transitioning from one state to another after performing a certain action. R is the reward function, used to evaluate the immediate reward the robot receives after performing a certain action in a given state. In this article, reward refers to the reward of the single step, and the average episode reward is described as return. γ is the discount factor, used to weigh the importance of future rewards.

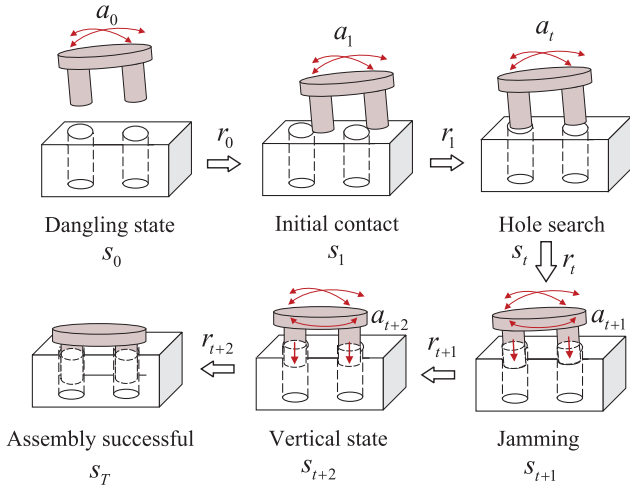


Fig. 2. Peg-in-hole assembly insertion process. The assembly peg is initially in the dangling state. In initial contact stage, the peg begins to contact the hole and is about to be inserted. In hole search stage, the robot carries the peg to find the location of the hole. In jamming stage, the peg partially enters the hole, but due to the orientation offset, it will get stuck in the hole. In vertical state and assembly success stage, the peg is adjusted to the vertical state and assembled successfully. When the robot is in state s_t , it performs action a_t , transitions to state s_{t+1} , and receives the reward r_t .

B. Continual Learning With Knowledge Sharing

In transfer learning, after the transfer process is completed, the model's skills on old tasks are not concerned typically. However, once skills for new tasks have been acquired, catastrophic forgetting often occurs in relation to the old tasks. In this article, we combine knowledge sharing and model acceleration to accelerate the learning of new tasks and avoid forgetting old tasks. Knowledge refers to the information that a robot encounters or accumulates during the assembly task. When robots perform different assembly tasks, each task interaction generates different types of data. The data include the robot's assembly process state s_t , the next action a_t , the reward received r_t , and the state at the next time step s_{t+1} .

When the robot learns a task, it generates interaction data with the environment. As shown in Fig. 3, a common experience pool is established to record the interaction data of each encountered task. Specifically, the interaction data with higher rewards are sampled into the common experience pool. The old task samples are replayed while learning a new task to alleviate forgetting. When the robot learns new tasks, the data in the common experience pool are reviewed every certain episodes. Therefore, while the robot is learning new tasks, it can also maintain the memory of the old tasks. As for the storage of the experience data, the maximum capacity of the common experience pool is set to 10^6 (The capacity can be set differently according to different experimental conditions). As the robot continuously learns new tasks, it stores 1000 groups of interaction data (s_t, a_t, r_t, s_{t+1}) in the common experience pool for each task. When it learns a new task, it reviews the interaction data in the common experience pool every 100 episodes, randomly selecting 200 groups of data each time. As a result, the

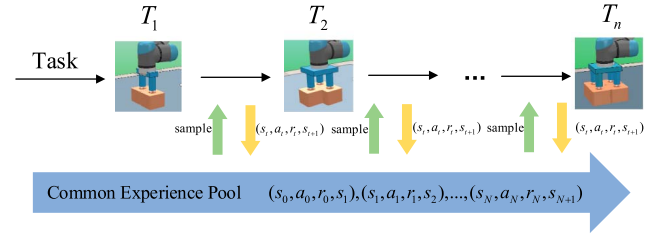


Fig. 3. Process of the continual learning method with knowledge sharing. During the learning process, the robot stores the interaction data of the new tasks into the common experience pool while sampling the data of the old tasks from the common experience pool to avoid forgetting.

size of the common experience pool will increase as the number of new tasks grows. Every time the robot learns a new task, 1000 groups of data will be added to the common experience pool. However, if the number of tasks continues to increase, when the amount of data exceeds the maximum capacity of the common experience pool, the data with low reward values in the common experience pool will be cleared out and the data with high reward values will be stored. Moreover, the data with the lowest reward value are eliminated globally across all tasks. Therefore, the capacity of the common experience pool can form a dynamic balance.

C. Continual Learning With Model Acceleration

The old model, which has acquired specific skills, is used to guide the new models to learn new skills quickly. Guiding the learning of new tasks through the knowledge acquired by previous models can give the robot a better initial state for the new task and accelerate the learning process. Unlike previous methods where the weights are set in advance; here, the weights of the old and new models are obtained through the network output. There is a certain implicit relationship between these weights and the types of tasks, as well as the differences between them, but the weights are not directly determined by the different types of tasks. Based on a predefined reward function, the network adjusts the weights of new and old strategies according to the performance on the task. It outputs different weights at each assembly step, enabling dynamic adaptation to different tasks and different assembly stages of the same task. The network adaptively learns the weights, allowing it to dynamically adjust the relationship between the old and new models throughout the learning process.

This article combines the old model and the new model into a combined model, or called strategy. Inside the combined model, the old model can guide the new model to learn and accelerate the learning process. At the same time, the new model includes the ability to learn new tasks and remember old tasks. Combining the two models into a new model can better achieve learning of multiple tasks and avoid forgetting. The number of models is very small, which does not impose a heavy burden on memory and computing. The formula for model-guided learning of new tasks is as follows, and the process is shown in Fig. 4:

$$\pi(s) = (1 - \rho) \cdot \pi^S(s) + \rho \cdot \pi^T(s) \quad (1)$$

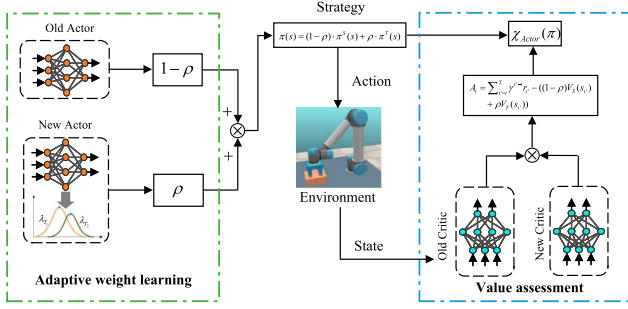


Fig. 4. Process of the continual learning method with model acceleration. The weighted sum of the output actions of the new strategy and the old strategy is used as the next action of the robot.

Algorithm 1: Continual Learning With Knowledge Sharing and Model Acceleration.

- 1: Get the output of the old model and the new model $o_t^{(i-1)}, o_t^{(i)}$
- 2: Get the add of the old model and the new model $o_t = (1 - \rho)o_t^{(i-1)} + \rho o_t^{(i)}$
- 3: Get the action of the robot, $a_t = o_t[1 : 6]$
- 4: Get the next action $a_t^{(i-1)}$ and a_t^i
- 5: **if** $ep\%20 == 0$ **then**
- 6: Store the interaction data into the common experience pool $\mathcal{B} \leftarrow (s_t, a_t, r_t, s_{t+1})$
- 7: **end if**
- 8: **if** $ep\%200 == 0$ **then**
- 9: Sample $(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{B}$
- 10: **end if**

where ρ is the weight of the new task domain strategy output, $\pi^S(s)$ is the old model, and $\pi^T(s)$ is the new model. The old model is used as guidance to facilitate the learning of a new model. Furthermore, the weights of the old and new models are adjusted in real time based on the network's output, dynamically adapting to changes in the task and environment.

In this article, the action space is continuous. The actions of the two strategies after weighting are still within the prescribed action space. However, if the action space is discrete, then we can weigh the probability of each discrete action. The result of weighting each action value is used as the final probability. The principle of continual learning with knowledge sharing and model acceleration is illustrated in Algorithm 1. As for the models used in the learning process, we only store the newest model and the model of the last task. At the same time, the models from previous tasks are abandoned and no longer in storage. In general, we only store a fixed number of models, so the memory is also fixed.

However, the continual learning framework proposed in this article is general and applicable to general reinforcement learning methods. This article uses the proximal policy optimization (PPO) algorithm as an example, serving as the base strategy for execution. PPO is a popular deep reinforcement learning algorithm within the actor-critic framework. To improve the

efficiency of the algorithm's use of interaction data, this article adds a replay buffer to store the interaction data between the robot and the environment in the order of interaction. However, the replay buffer is where the robot stores data for each task during the interaction process. The common experience pool stores interaction data from the robots as they perform various tasks. Unlike a standard replay buffer, which stores interaction data from learning a single task, the common experience pool stores selected data that are beneficial for model learning. Each task has a new replay buffer for storing data. This is different from the common experience pool, which is established for all tasks. With this replay buffer, PPO can sample data from it and update the network. The actor network uses the probability of the output action to measure the improvement of the new strategy relative to the old strategy as follows:

$$\kappa_t(\theta) = \frac{\rho \cdot \phi_\pi^T(a_t | s_t) + (1 - \rho) \cdot \phi_\pi^S(a_t | s_t)}{\rho \cdot \phi_{\pi_{old}}^T(a_t | s_t) + (1 - \rho) \cdot \phi_{\pi_{old}}^S(a_t | s_t)} \quad (2)$$

where s_t represents the robot state at the current time, a_t represents the next action, S is the source domain, T is the target domain, $\phi_\pi^T(a_t | s_t)$ and $\phi_{\pi_{old}}^T(a_t | s_t)$ are the new and old strategies in the target domain, and $\phi_\pi^S(a_t | s_t)$ and $\phi_{\pi_{old}}^S(a_t | s_t)$ represent the new and old strategies in the source domain, respectively. The new strategy and old strategy refer to the strategies after and before a single update step in PPO, while the new model and old model in (1) refer to the models learned for the new and old tasks, respectively. ρ is the same as that in (1). $\kappa_t(\theta)$ represents the degree of preference of the new strategy compared to the old strategy for a particular action a_t . If $\kappa_t(\theta) > 1$, it means the new strategy is more inclined to choose action a_t . If $\kappa_t(\theta) < 1$, it means the new strategy has reduced its preference for action a_t . Then, the advantage function is calculated

$$A_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'} - ((1 - \rho)V_S(s_{t'}) + \rho V_T(s_{t'})) \quad (3)$$

where γ denotes the discount factor. $V_S(s_{t'})$ represents the output of the source domain critic network, and $V_T(s_{t'})$ represents the output of the target domain critic network. The loss function is calculated as follows:

$$\chi_{Actor}(\pi) = E_t[\min(\kappa_t(\theta_{Actor})A_t, \text{clip}(\kappa_t(\theta_{Actor}), 1 - d, 1 + d)A_t)] \quad (4)$$

where $\text{clip}(\kappa_t(\theta_{Actor}), 1 - d, 1 + d)$ denotes a function that restricts the value $\kappa_t(\theta_{Actor})$ to the interval $[1 - d, 1 + d]$. θ_{Actor} represents the actor network parameter.

Then, the actor network parameters can be updated

$$\theta_{Actor} \leftarrow \theta_{Actor} - \eta_{Actor} \nabla_{\theta_{Actor}} \chi_{Actor}(\theta_{Actor}) \quad (5)$$

where η_{Actor} indicates the learning rate of the actor network.

The loss function of the critic network is as follows:

$$\chi_{Critic}(\pi) = E_t[(R_t - (\rho \cdot V_T(s_t) + (1 - \rho) \cdot V_S(s_t)))^2]. \quad (6)$$

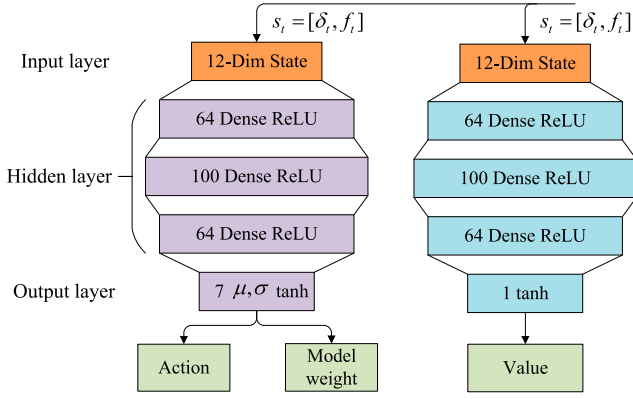


Fig. 5. Structure of the networks. The left is the actor network, which inputs 12-dimensional state and gets the robot next action and model weight. The right is the critic network, which also inputs 12-dimensional state and gets the value of the state.

ρ is the same as that in (1). The critic network parameter can be updated as follows:

$$\theta_{\text{Critic}} \leftarrow \theta_{\text{Critic}} - \eta_{\text{Critic}} \nabla_{\theta_{\text{Critic}}} \chi_{\text{Critic}}(\theta_{\text{Critic}}) \quad (7)$$

where θ_{Critic} represents the critic network parameter and η_{Critic} indicates the learning rate of the critic network. The method proposed in this article is based on the traditional reinforcement learning method PPO, focusing on addressing the issues of transfer and continual learning between different types of tasks. By introducing a continual learning mechanism during the learning of different tasks, it can accelerate the learning of new tasks while maintaining efficient memory of old tasks. The structure of the networks is shown in Fig. 5.

First, the learning process consists of two loops, 1000 assembly episodes, and each assembly episode contains a maximum of 45 steps. The input state data are 12-dimensional in the algorithm, including the robot end 6-D position and orientation as well as the robot end 6-D force/torque. The middle layers contain 64 neurons, 100 neurons, and 64 neurons, respectively. The output includes the robot next action and model weight. The actions are the mean and standard deviation of the normal distribution $N \sim (\mu, \sigma)$. Among them, μ and σ , respectively, contain 6-D data. The dimensions of the input layer, middle layers, and output layer of the algorithm are 12, 64, 100, 64, and 7, respectively. Therefore, the output of the combined strategy includes 7-D data, consisting of the 6-D motion of the robot end effector and the weight of the new model. Second, for the network structure used in this article, it contains actor network and critic network. Actor network includes the input layer (12 dimensions), the first hidden layer (64 neurons), the second hidden layer (100 neurons), the third hidden layer (64 neurons), and the output layer (7 dimensions). The total number of parameters is $(12 \times 64) + (64 \times 100) + (100 \times 64) + (64 \times 7) = 14016$. Critic network includes the input layer (12 dimensions), the first hidden layer (64 neurons), the second hidden layer (100 neurons), the third hidden layer (64 neurons), and the output layer (1 dimension). The total number of parameters is $(12 \times 64) + (64 \times 100) + (100 \times 64) + (64 \times 1) = 13632$. The force controller takes the difference between the expected force and the

Algorithm 2: Strategy Learning and Update.

Input: Current state $s_t = [\delta_t, f_t]$

Output: $a_t = [\Delta x_t, \Delta y_t, \Delta z_t, \Delta \alpha_t, \Delta \beta_t, \Delta \gamma_t, \rho]$, represents the robot next action, and the weight of the new strategy and the old strategy

- 1: Initialize the force control strategy.
- 2: **for** each task i **do**
- 3: Initialize the old strategy $\pi^{i-1}(s)$ and the common experience pool $\mathcal{B}$
- 4: **for** each episode ep **do**
- 5: Initialize the current strategy $\pi^{(i)}(s)$
- 6: Obtain the initial state of the robot
- 7: **for** each step t **do**
- 8: Execute the interactive process in Algorithm 1
- 9: Use A_t to update the Critic Network
- 10: Use $\chi_{\text{Actor}}(\pi)$ to update the Actor Network
- 11: **end for**
- 12: **end for**
- 13: **end for**

actual contact force at the end of the robot, and outputs action, as depicted in [37]. The overall process of the strategy learning and update is expressed in Algorithm 2.

D. Reward Design

Fuzzy reward system is designed in this article to minimize the assembly steps and contact forces/torques. The reward function is defined as follows:

$$r = \omega_1 r_1 + \omega_2 r_2 \quad (8)$$

where r_1 refers to the positive reward, and r_2 refers to the negative reward during the assembly process

$$r_2 = -k/k_{\max} \quad (9)$$

where k is the current assembly step, and $k_{\max}$ is the maximum assembly step allowed. This implies that a larger k results in a smaller reward value.

The positive reward r_1 is obtained by the fuzzy reward system, which features a two-layer fuzzy logic structure, as depicted in Fig. 6. The first layer consists of two fuzzy logic systems: position fuzzy logic system and force/torque fuzzy logic system. The position fuzzy logic system inputs the current assembly depth and assembly action, while the force/torque fuzzy logic system processes the contact forces and torques. The output from the first layer feeds into the second layer, which then outputs the final reward value. The continuous characteristic parameters are processed through the fuzzy control system to determine the output space. The design goal of the reward function in this article is to guide the behavior of the robot in the following two aspects.

- 1) *Minimize the Number of Assembly Steps (Negative Reward Term):* It encourages quick completion of tasks and avoids long assembly steps.
- 2) *Control the Contact Force/Torque (Positive Reward Term):* The fuzzy logic system dynamically rewards safe

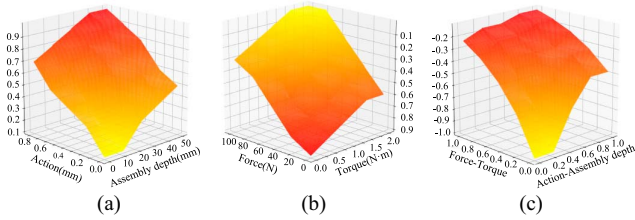


Fig. 6. Output value of each fuzzy layer. (a) Assembly action-depth fuzzy output. (b) Force-torque fuzzy output. (c) Fuzzy output of the second layer.

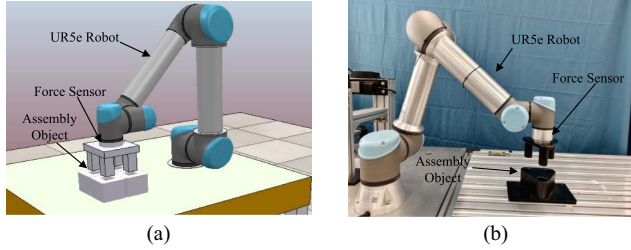


Fig. 7. Assembly experimental platform. (a) Simulation. (b) Real-world.

and efficient assembly actions to avoid excessive contact forces that damage the workpiece or robot.

IV. SIMULATION

To verify the advancement of the proposed method with multiple tasks, the environment containing various peg-in-hole assembly tasks is established in CoppeliaSim, as shown in Fig. 7. The assembly tasks contain single, dual, triple, and four peg-in-hole assembly tasks for both circle and square. To simulate the positioning errors of the camera, this article sets the position offset of the peg relative to the hole to be within 1 mm in the x and y directions, and the orientation offset to be within 3° .

A. Direct Learning and Weight Adjustment

In this section, the various methods with weight adjustment are obtained compared with that without weight adjustment. The proposed method leverages the old model to expedite the learning of new tasks, with the strategy network managing the weight adjustments of both models.

To demonstrate the advantages of our approach, we compare it against direct learning and the policy fusion transfer (PFT) method in [38]. As shown in Fig. 8, the proposed method achieves a higher initial return, faster convergence speed, and a higher final return than the other methods. Continual learning is a two-way process that includes the transfer to new tasks and the transfer to old tasks, that is, the memory of old tasks. However, transfer learning refers to the process of transferring to new tasks, and it does not contain the memory of old tasks. The newly learned task strategies have limited adaptability to old tasks, while the proposed method in this article not only demonstrates strong learning capabilities in unidirectional task transfer but also exhibits excellent performance in bidirectional

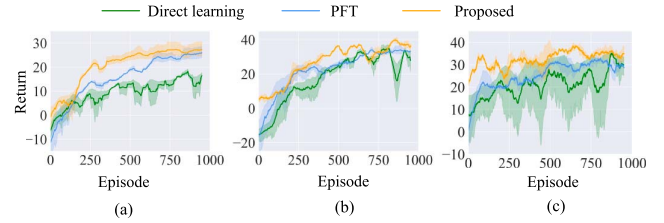


Fig. 8. Training results of different tasks with direct learning and various weight adjustment methods in simulation environment. (a) Four peg and hole. (b) Dual peg and hole. (c) Single peg and hole.

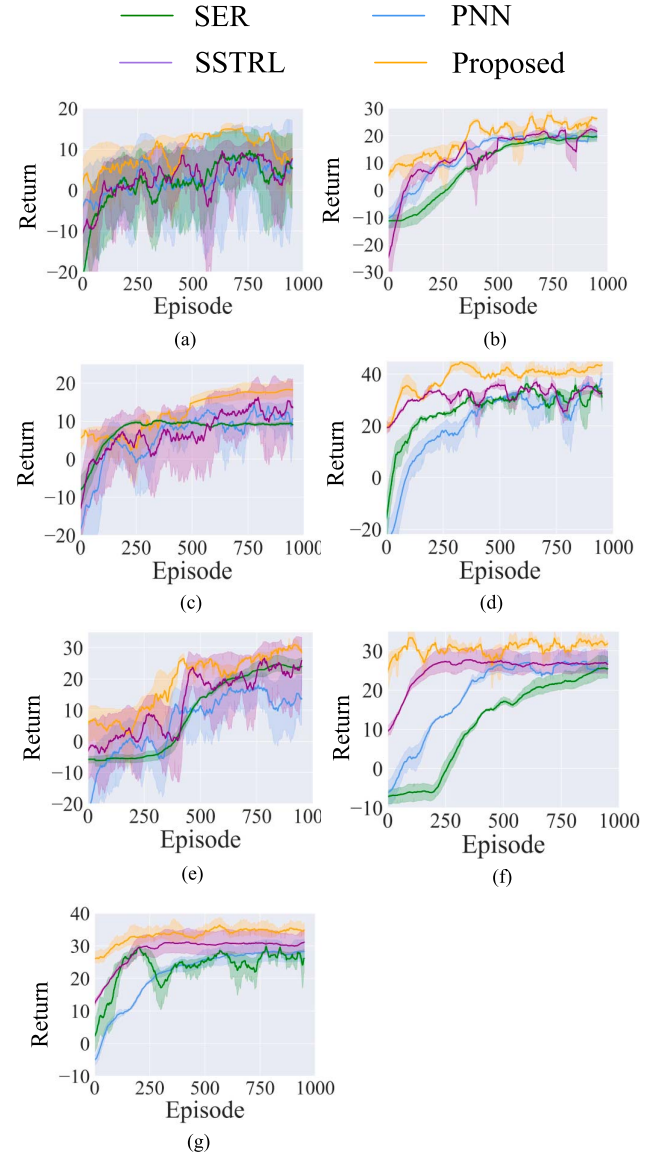


Fig. 9. Training results of different tasks with three continual learning methods in simulation environment. (a) Square and four. (b) Circular and triple. (c) Circular and four. (d) Square and single. (e) Square and dual. (f) Circular and dual. (g) Circular and single.

transfer. PFT is a transfer learning process, as a result, the proposed method performs better than PFT in the two-way process.

TABLE I
INSERTION SUCCESS RATE IN SIMULATION

	SER in [26]	PNN in [33]	SSTRL in [30]	Proposed
Circular and Single	89.2%	88.8%	88.8%	90.4%
Circular and Dual	84.8%	86.8%	89.2%	91.2%
Circular and Triple	96.4%	99.2%	98.8%	100%
Circular and Four	92.4%	95.2%	94.8%	97.2%
Square and Single	78.8%	89.6%	92.0%	96.0%
Square and Dual	83.6%	86.4%	92.4%	94.8%
Square and Triple	98.0%	100%	99.6%	100%
Square and Four	84.0%	86.0%	86.8%	88.4%

Note: Bold entries represents the highest assembly success rate for a specific assembly task.

B. Comparison With Continual Learning Methods

In addition, we also add experiments to compare with currently popular continual learning methods. We compare selective experience replay (SER) in [26], PNNs in [33], and self-supervised task representation learning (SSTRL) in [30] with the proposed method. Meanwhile, to verify the performance of the proposed method on sequential assembly tasks, eight assembly tasks are used for testing, including circular and single, circular and dual, circular and triple, circular and four, square and single, square and dual, square and triple, and square and four peg-in-hole tasks. We leverage the four methods to learn sequentially on the eight peg-in-hole assemblies. Among them, the first task is to use the PPO model to learn independently, and the other seven tasks are learned using four continual learning methods. The return curves during the learning process are shown in Fig. 9. This demonstrates that the proposed method still has higher learning efficiency compared with current continual learning methods.

In addition, each method is tested on the eight tasks. Each method on one task contains five groups, and each group contains 50 episodes. The test success rates are shown in Table I. As can be seen from the table, the success rates of the proposed method on all the tasks are higher than those of other methods, which also verify its memory of the tasks that have been learned.

From the experiments above, it can be seen that the proposed method not only exhibits high learning efficiency during the training phase but also achieves the highest test success rate during the testing phase, thus verifying its adaptability and generalization capabilities across multiple tasks.

V. REAL-WORLD EXPERIMENT

To verify that the strategy obtained in simulation can also achieve excellent performance in the real world, this article builds a flexible assembly platform in the real world. As shown in Fig. 10, the experimental platform consists of a UR5e robot, a 6-D force sensor, and the assembly objects. The sequential assembly tasks used in this section include triple peg and hole, dual peg and hole, single peg and hole, circular peg and hole, square peg and hole, and triangular peg and hole, as well as VGA and USB.

To verify the sim-to-real transfer performance of the proposed method, the strategies used in this section are all directly

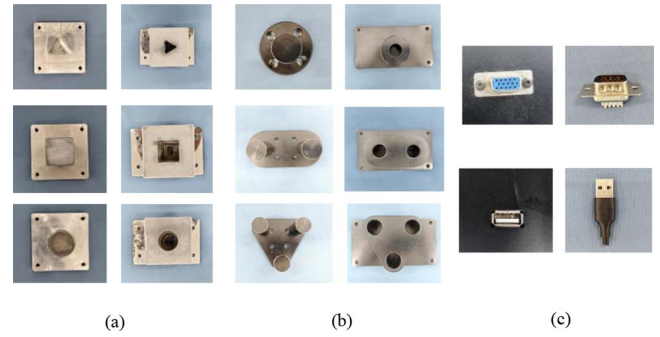


Fig. 10. Assembly objects. (a) Different shapes of pegs and holes. (b) Multiple pegs and holes. (c) Electrical connectors.

TABLE II
INSERTION SUCCESS RATE IN REAL WORLD

	SER in [26]	PNN in [33]	SSTRL in [30]	Proposed
Circular	89.6%	90.4%	93.6%	95.2%
Square	98.0%	100%	99.6%	100%
Triangular	98.4%	100%	100%	100%
Single	82.4%	85.6%	84.8%	90.4%
Dual	98.8%	96.4%	99.6%	100%
Triple	94.4%	95.6%	96.4%	100%
USB	73.6%	75.6%	80.4%	86.0%
VGA	90.2%	94.4%	95.6%	99.2%

Note: Bold entries represents the highest assembly success rate for a specific assembly task.

trained in simulation and are not retrained in the real world. In addition, as shown in Fig. 10(a) and (b), the single peg and hole, multiple peg and hole used in this article have different colors. This is because the proposed assembly strategy is independent of the color of the assembly objects. In the initial assembly stage, this article assumes a certain offset in position and orientation between the peg and the hole to simulate camera positioning errors. After the peg and hole come into contact, the network input only includes the robot end effector pose and contact force and torque. Therefore, during assembly, contact information is considered more, regardless of the color of the assembled objects.

A. Single Peg-in-Hole Assembly With Different Shapes

In this section, single peg-in-hole assembly experiments with different shapes are conducted. The assembly objects include circular, square, triangular pegs, and holes. The assembly clearance between the peg and the hole is maintained at 0.5 mm. The position offset of the peg in the horizontal direction is randomly within ± 1 mm, and the orientation offset is randomly within $\pm 3^\circ$. Each task is tested with five groups and each group contains 50 episodes. The maximum assembly step is 45. The test success rates of the three single peg-in-hole assembly tasks are shown in Table II. A comparative analysis of the table reveals that the proposed method achieves a consistent improvement in success rate over the three comparative methods.

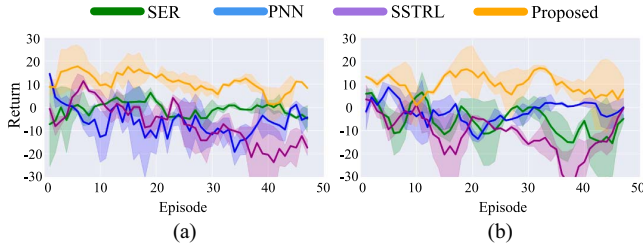


Fig. 11. Return during the assembly process. (a) Dual peg-in-hole assembly. (b) Triple peg-in-hole assembly.

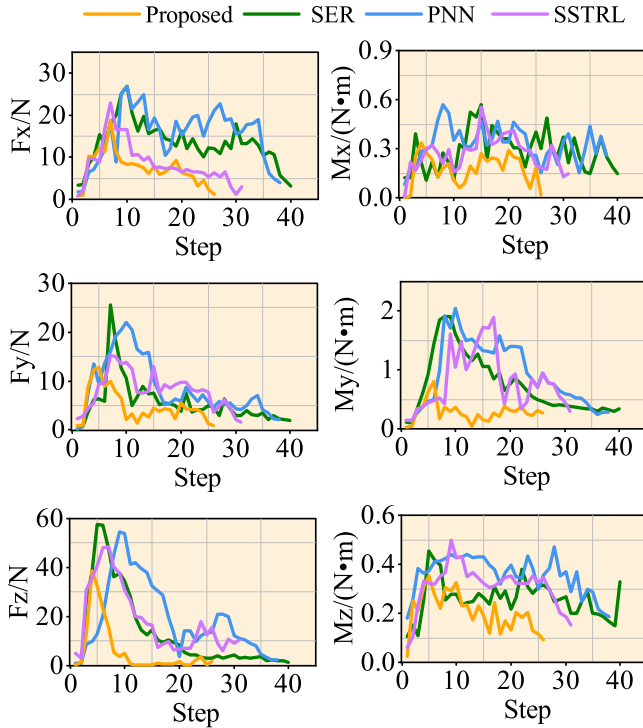


Fig. 12. Real-time contact forces and torques during the VGA assembly process.

B. Multiple Peg-in-Hole Assembly

This section conducts multiple peg-in-hole assembly experiments with different numbers of pegs and holes. The multiple peg-in-hole assembly tasks include triple, dual, and single peg and hole. Their assembly clearance is 0.2 mm. The position offset of the peg in the horizontal direction is randomly within ± 0.5 mm, and the orientation offset is randomly within $\pm 2^\circ$.

The test success rates of the three multiple peg-in-hole assembly tasks are shown in Table II. For the dual peg-in-hole assembly and triple peg-in-hole assembly, the success rate of the proposed method achieves 100%, which is higher than the other three methods. For the single peg-in-hole assembly, the success rate of the proposed method increases by up to 8%. In addition, the returns during the assembly process are also recorded and plotted as in Fig. 11. A comparative analysis of Fig. 11 reveals a marked increase in the return achieved by the proposed method throughout the assembly process relative to the other approaches.

C. Electrical Connector Assembly

In this section, the complex electrical connector assembly experiments are conducted. The assembly objects include USB and VGA. The orientation offset of the USB in the horizontal direction is randomly within $\pm 1^\circ$ and the VGA is randomly within $\pm 0.5^\circ$. The test success rates of the electrical connector assembly tasks are shown in Table II. This indicates that the proposed method still has excellent generalization ability when generalizing to unseen complex assembly tasks. In addition, this also demonstrates that the proposed method can effectively avoid catastrophic forgetting when faced with multitask sequential learning. The real-time contact forces and torques during the VGA assembly process are also recorded and plotted as in Fig. 12. This demonstrates that in one assembly episode, the contact forces and torques of the proposed method are smaller than those of other methods most of the time. As can be seen from the figure, the contact force of the robot is zero during the initial dangling state. Then, it quickly enters the initial contact phase. Because the robot is moving downward during this phase, the contact force gradually increases until the hole is successfully located. Subsequently, the insertion task is completed through the combined action of the network strategy and force control.

The experimental results above demonstrate that the proposed method not only possesses strong transfer capabilities across various tasks but also performs exceptionally well in sim-to-real transfer scenarios.

VI. CONCLUSION

In this article, a hybrid continual learning method of knowledge sharing and model acceleration is presented. Based on the established common experience pool, this approach is able to avoid forgetting by reviewing the old task data. By the guidance of the old strategy, the learning of new task strategies can be accelerated. At the same time, the balance between the old and new strategies is adaptively achieved through interaction with the environment.

The current common experience pool mechanism relies on a fixed selection strategy. Future research can explore adaptive selection schemes, such as sampling based on task relevance, to optimize storage utilization and data utilization efficiency in long-term task sequences. Another promising direction is to introduce meta-continual learning strategies to enable robots to autonomously identify common substructures between tasks and generalize more effectively to unknown assembly scenarios.

REFERENCES

- [1] Y. Chen et al., "Embodied AI: A survey on the evolution from perceptive to behavioral intelligence," *Smart Mater. Biomed. Appl.*, vol. 1, no. 3, 2025, Art. no. e70003.
- [2] M. Liu, L. He, and M. Shang, "Dynamic neural network for bicriteria weighted control of robot manipulators," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 8, pp. 4570–4583, Aug. 2023.
- [3] H. Huang, L. Jin, and Z. Zeng, "A momentum recurrent neural network for sparse motion planning of redundant manipulators with majorization-minimization," *IEEE Trans. Ind. Electron.*, vol. 72, no. 12, pp. 13532–13541, Dec. 2025.

- [4] S. Li, Z. Yan, Z. Wang, and Y. Gao, "VLM-MSGraph: Vision language model-enabled multi-hierarchical scene graph for robotic assembly," *Robot. Comput.-Integr. Manuf.*, vol. 94, 2025, Art. no. 102978.
- [5] D. Li, X. Li, H. Zhao, D. Ge, and H. Ding, "Robotic multiple peg-in-hole assembly with deviation estimation and fast insertion control," *IEEE Trans. Ind. Electron.*, vol. 72, no. 1, pp. 670–680, Jan. 2025.
- [6] D. Li, H. Zhao, D. Ge, X. Li, and H. Ding, "A novel robotic multiple peg-in-hole assembly pipeline: Modeling, strategy, and control," *IEEE/ASME Trans. Mechatron.*, vol. 29, no. 4, pp. 2602–2613, Aug. 2024.
- [7] J. Su, C. Liu, and R. Li, "Robot precision assembly combining with passive and active compliant motions," *IEEE Trans. Ind. Electron.*, vol. 69, no. 8, pp. 8157–8167, Aug. 2022.
- [8] Z. Zhang et al., "Hybrid force/position control of multi-mobile manipulators for cooperative operation without force measurements," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 71, no. 1, pp. 397–410, Jan. 2024.
- [9] M. Liu, F. Zhang, L. He, and M. Shang, "Dynamic neural network for motion/force control of manipulators with polynomial noises," *IEEE Trans. Ind. Electron.*, vol. 71, no. 10, pp. 12559–12569, Oct. 2024.
- [10] Y. You, P. Xu, and B. Li, "Robotic compliant assembly strategy for radio frequency connectors with initial positioning errors," *IEEE Robot. Autom. Lett.*, vol. 10, no. 11, pp. 12024–12031, Nov. 2025.
- [11] Y. Mei, R. Li, Y. Tan, D. Zhu, T. Li, and Z. Zhou, "Robotic assembly strategy with wrist force sense for narrow clearance peg-in-hole," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 8709–8724, 2025.
- [12] Z. Hou, Z. Li, C. Hsu, K. Zhang, and J. Xu, "Fuzzy logic-driven variable time-scale prediction-based reinforcement learning for robotic multiple peg-in-hole assembly," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 1, pp. 218–229, Jan. 2022.
- [13] K. Tanaka, R. Yonetani, M. Hamaya, R. Lee, F. v Drigalski, and Y. Ijiri, "TRANS-AM: Transfer learning by aggregating dynamics models for soft robotic assembly," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 4627–4633.
- [14] S. Li, P. Zheng, J. Fan, and L. Wang, "Toward proactive human-robot collaborative assembly: A multimodal transfer-learning-enabled action prediction approach," *IEEE Trans. Ind. Electron.*, vol. 69, no. 8, pp. 8579–8588, Aug. 2022.
- [15] L. Jin, L. Wei, and S. Li, "Gradient-based differential neural-solution to time-dependent nonlinear optimization," *IEEE Trans. Autom. Control*, vol. 68, no. 1, pp. 620–627, Jan. 2023.
- [16] T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, and N. Díaz-Rodríguez, "Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges," *Inf. Fusion*, vol. 58, pp. 52–68, 2020.
- [17] Q. Liu, Z. Ji, W. Xu, Z. Liu, B. Yao, and Z. Zhou, "Knowledge-guided robot learning on compliance control for robotic assembly task with predictive model," *Expert Syst. Appl.*, vol. 234, Jul. 2023, Art. no. 121037.
- [18] A. Zhu, T. Dai, G. Xu, P. Pauwels, B. De Vries, and M. Fang, "Deep reinforcement learning for real-time assembly planning in robot-based prefabricated construction," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 3, pp. 1515–1526, Jul. 2023.
- [19] Z. Hou, J. Fei, Y. Deng, and J. Xu, "Data-efficient hierarchical reinforcement learning for robotic assembly control applications," *IEEE Trans. Ind. Electron.*, vol. 68, no. 11, pp. 11565–11575, Nov. 2021.
- [20] Y. Shi et al., "Proactive action visual residual reinforcement learning for contact-rich tasks using a torque-controlled robot," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 765–771.
- [21] J. Song, Q. Chen, and Z. Li, "A peg-in-hole robot assembly system based on gauss mixture model," *Robot. Comput.-Integr. Manuf.*, vol. 67, 2021, Art. no. 101996.
- [22] X. Deng, J. Liu, H. Gong, H. Gong, and J. Huang, "A human-robot collaboration method using a pose estimation network for robot learning of assembly manipulation trajectories from demonstration videos," *IEEE Trans. Ind. Inform.*, vol. 19, no. 5, pp. 7160–7168, May 2023.
- [23] S. Auddy, J. Hollenstein, M. Saveriano, A. Rodríguez-Sánchez, and J. Piater, "Continual learning from demonstration of robotics skills," *Robot. Auton. Syst.*, vol. 165, 2023, Art. no. 104427.
- [24] M. Riemer, T. Klinger, D. Bouneffouf, and M. Franceschini, "Scalable recollections for continual lifelong learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, pp. 1352–1359, 2019.
- [25] F. Benzing, "Unifying importance based regularisation methods for continual learning," in *Proc. Int. Conf. Artif. Intell. Stats.*, 2022, pp. 2372–2396.
- [26] D. Isele and A. Cosgun, "Selective experience replay for lifelong learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 404, no. 8, pp. 3302–3309, 2018.
- [27] X. Li and W. Wang, "GopGAN: Gradients orthogonal projection generative adversarial network with continual learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 1, pp. 215–227, Jan. 2023.
- [28] Y. Lee et al., "Composing complex skills by learning transition policies," in *Proc. 36th Int. Conf. Learn. Represent.*, 2019, pp. 1–19.
- [29] Y. Lee et al., "Adversarial skill chaining for long-horizon robot manipulation via terminal state regularization," in *Proc. Conf. Robot. Learn.*, 2022, pp. 406–416.
- [30] M. B. Hafez and S. Wermter, "Continual robot learning using self-supervised task inference," *IEEE Trans. Cogn. Dev. Syst.*, vol. 16, no. 3, pp. 947–960, Jun. 2024.
- [31] J. Schwarz et al., "Progress & compress: A scalable framework for continual learning," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 4528–4537.
- [32] X. Zhao, H. Zhao, P. Chen, and H. Ding, "Model accelerated reinforcement learning for high precision robotic assembly," *Int. J. Intell. Robot. Appl.*, vol. 4, pp. 202–216, 2020.
- [33] A. A. Rusu et al., "Progressive neural networks," 2016, *arXiv:1606.04671*.
- [34] P. Sprechmann et al., "Memory-based parameter adaptation," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1–16.
- [35] C. Gao, H. Gao, S. Guo, T. Zhang, and F. Chen, "CRIL: Continual robot imitation learning via generative and prediction model," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2021, pp. 6747–6754.
- [36] D. Maltoni and V. Lomonaco, "Continuous learning in single-incremental-task scenarios," *Neural Netw.*, vol. 116, pp. 56–73, 2019.
- [37] L. Jin, Y. Men, R. Song, F. Li, Y. Li, and X. Tian, "Robot skill generalization: Feature-selected adaptation transfer for peg-in-hole assembly," *IEEE Trans. Ind. Electron.*, vol. 71, no. 3, pp. 2748–2757, Mar. 2024.
- [38] Y. Men, L. Jin, T. Cui, Y. Bai, F. Li, and R. Song, "Policy fusion transfer: The knowledge transfer for different robot peg-in-hole insertion assemblies," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–10, Aug. 2023.



Ligang Jin received the B.S. degree in automation engineering from Wuhan University, Wuhan, China, in 2019, and the Dr.Eng. degree in electronic information from the School of Control Science and Engineering, Shandong University, Jinan, China, in 2025.

He is currently a Postdoctoral Researcher with the School of Artificial Intelligence, Shandong University. His research interests include robot learning and embodied intelligence.



Chaoqun Wang received the B.E. degree in automation from Shandong University, Jinan, China, in 2014, and the Ph.D. degree in robotics and artificial intelligence from the Department of Electronic Engineering, The Chinese University of Hong Kong, Hong Kong, China, in 2019.

He is currently a Professor with the School of Control Science and Engineering, Shandong University. During his Ph.D. study, he spent six months as a Visiting Scholar with The University of British Columbia, Vancouver, BC, Canada. He

was a Postdoctoral Fellow with the Department of Electronic Engineering, The Chinese University of Hong Kong, from 2019 to 2020. His research interests include autonomous vehicles, active and autonomous exploration, and path planning.



Zhiwei Cui (Member, IEEE) received the master's degree from Beihang University, Beijing, China, in 2016, and the Ph.D. degree from Tsinghua University, Beijing, in 2020, both in mechanical engineering.

He is currently a Professor with the School of Artificial Intelligence, Shandong University, Jinan, China. From 2021 to 2025, he was a Postdoctoral Fellow with Chow Yuk Ho Technology Centre for Innovative Medicine, The Chinese University of Hong Kong, Hong Kong, China.

His research interests include cable-driven robotics, industrial robotics, medical robotics, and neural networks.



Xincheng Tian received the B.S. and M.S. degrees in industrial automation from the original Shandong Industrial University (now Shandong University), Jinan, China, in 1988 and 1993, respectively, and the Dr.Eng. degree in guidance, control, and simulation from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2000.

He is currently a Professor with the School of Control Science and Engineering, Shandong University. His research interests include the robot control technology, intelligent control of electromechanical systems, and computer control technology and applications.



Yibin Li (Member, IEEE) received the B.S. degree in automation engineering from Tianjin University, Tianjin, China, in 1982, the M.S. degree in electrical engineering from Shandong University of Science and Technology, Jinan, China, in 1990, and the Dr.Eng. degree in automation engineering from Tianjin University, in 2008.

He is currently a Professor with the School of Control Science and Engineering, Shandong University, Jinan. His research interests include intelligent robots, special robots, intelligent vehicles, intelligent control, and mechatronics.



Fuchun Sun (Fellow, IEEE) received the Ph.D. degree in computer science from Tsinghua University, Beijing, China, in 1997.

He is currently a Professor with the Department of Computer Science and Technology, Tsinghua University, and the School of Artificial Intelligence, Shandong University, Jinan, China. His research interests include robotic perception and embodied intelligence.

Dr. Sun was a recipient of the National Science Fund for Distinguished Young Scholars.



Rui Song (Senior Member, IEEE) received the B.S. degree in industrial automation and the M.S. degree in control theory and engineering from Shandong University of Science and Technology, Jinan, China, in 1998 and 2001, respectively, and the Dr.Eng. degree in control theory and engineering from Shandong University, Jinan, in 2011.

He is currently a Professor with the School of Control Science and Technology, Shandong University. His research interests include intelligent robots and control theory.